

Same Language
Bigger Opportunities

"A Strong Foundation
Builds Amazing Developers"
...

JAVA

COMPLETE NOTES

PART 1

FROM ZERO → STRONG JAVA FUNDAMENTALS

8 pages that build your Java foundation step-by-step

Learn
Code
Grow
Repeat
...



Write
Compile
Run
Anywhere
...



Small Steps
Big Progress



Better Code
Brighter Future

WHAT YOU'LL LEARN ↓

- ✓ Java & JVM fundamentals
- ✓ Variables & Data Types
- ✓ Operators
- ✓ Control Statements
- ✓ Methods & Arrays
- ✓ Strings
- ✓ Classes & Objects
- ✓ Constructors & this

☐ Build Skills

☐ Solve Problems

☐ Crack Interviews

☐ Grow Your Career

"Consistency Turns Learners into Developers"



Don't skip the basics

Strong Java developers don't memorize syntax. They understand what happens behind it.

"Good Developers Never Stop Learning"

One Page Closer to Your Goals

SWIPE →

Page 2 starts the real Java journey...

WAIT FOR PART 2

Classes → Inheritance → Polymorphism
→ Abstraction → Encapsulation

That's where Java OOP gets interesting.

SAVE THIS SERIES

Perfect for beginners + interview revision



1. JAVA INTRODUCTION + FEATURES

① What is Java?

- Java is a **high-level**, **class-based**, **object-oriented** programming language.
- Java was developed by **James Gosling** and his team at **Sun Microsystems**.
- Java was first released in **1996**.
- Now Java is owned by **Oracle Corporation**.
- Java is designed to be as **platform independent** as possible.

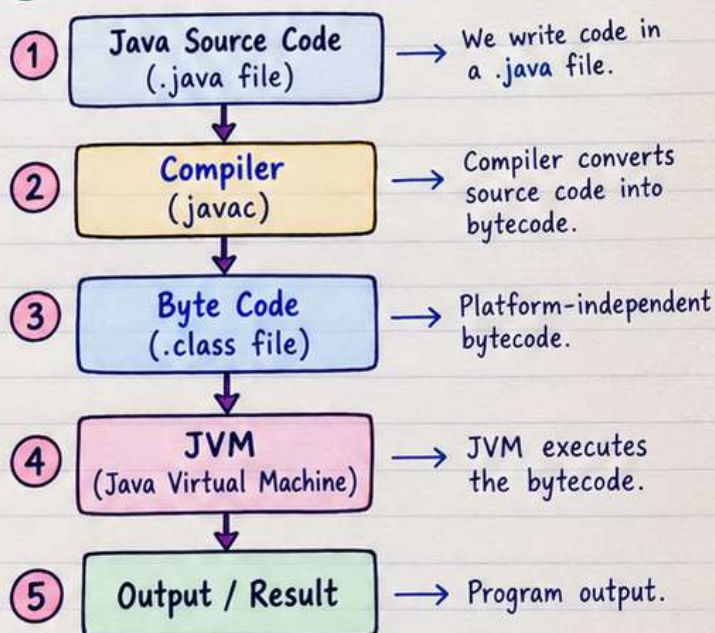
② Why Java?

- Java was designed with the goal of being **platform independent**, **secure** and **portable**.
- It is widely used in software development, web applications, Android applications, enterprise systems and more.

Java Program Example

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

③ How Java Works?



④ Features of Java

- 1 **Simple** - Easy to learn and understand.
- 2 **Object-Oriented** - Based on OOP concepts like class, object, inheritance etc.
- 3 **Platform Independent** - Write once, run anywhere.
- 4 **Secure** - Provides features such as controlled memory access and bytecode verification.
- 5 **Robust** - Strong memory management and exception handling.
- 6 **Portable** - Same bytecode can run on different platforms with a compatible JVM.
- 7 **Architecture Neutral** - No dependency on a specific processor architecture.
- 8 **High Performance** - JIT compiler improves runtime performance.
- 9 **Multithreaded** - Supports concurrent execution using multiple threads.
- 10 **Distributed** - Supports building distributed applications.
- 11 **Dynamic** - Classes can be loaded dynamically at runtime.
- 12 **Compiled + Interpreted** - Java source code is compiled into bytecode and the JVM executes it.
- 13 **Automatic Memory Management** - Garbage Collector manages unused objects.
- 14 **Rich Standard Library** - Provides many built-in APIs for application development.

★ KEY IDEA

Java is:

Simple + Secure + Portable + Robust + Object-Oriented + Platform Independent

🎯 INTERVIEW SURPRISE

Q: Why is Java called platform independent?

Answer: Java source code is compiled into platform-independent bytecode. The JVM on each operating system executes that bytecode.

BYTECODE → PLATFORM INDEPENDENT

JVM → PLATFORM SPECIFIC IMPLEMENTATION

★ Remember

- Java follows WORA - Write Once, Run Anywhere.
- .java → Source Code
- javac → Compiler
- .class → Bytecode
- JVM → Executes Bytecode

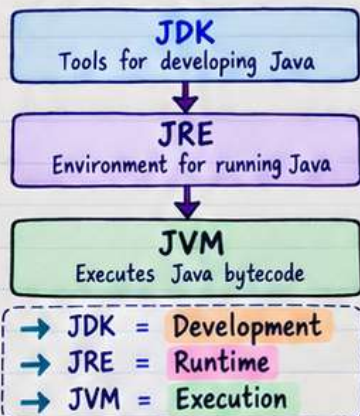
2. JVM, JRE & JDK

Understand how Java is developed, packaged and executed.

① The Big Picture

Think of Java development as three layers:

JDK → JRE → JVM



② JDK - Java Development Kit

The JDK is the toolkit a developer uses to create Java applications.

- ✓ Write Java source code
- ✓ Compile source code
- ✓ Run applications
- ✓ Debug and package programs
- ✓ Provides development utilities



COMMON TOOLS

- **javac** → compiler
- **java** → launches a Java application
- **javadoc** → generates documentation
- **jar** → packages Java applications/classes

③ JRE - Java Runtime Environment

The JRE provides the runtime environment required for a Java application to execute.

- ✓ Provides JVM
- ✓ Provides required runtime libraries
- ✓ Supports application execution
- ✓ Does not provide the full development toolkit
- ✓ Used to **RUN Java applications**



④ JVM - Java Virtual Machine

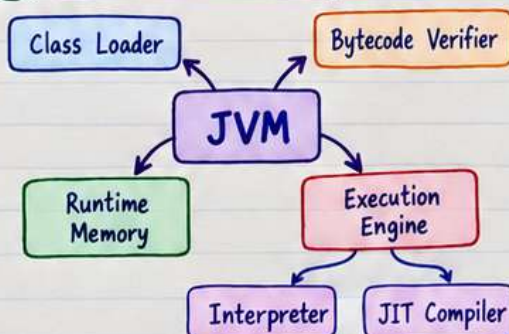
The JVM is the runtime engine that loads and executes Java bytecode.

- ✓ Loads classes
- ✓ Verifies bytecode
- ✓ Executes bytecode
- ✓ Manages runtime memory



♂ JVM is the part that actually **executes** the bytecode.

⑤ JVM - Main Components



- 1 **Class Loader**
→ Loads required class files.
- 2 **Bytecode Verifier**
→ Checks bytecode before execution.
- 3 **Runtime Memory**
→ Provides memory areas required during execution.
- 4 **Execution Engine**
→ Executes the loaded bytecode.
- 5 **JIT Compiler**
→ Compiles frequently executed code into native machine code at runtime.

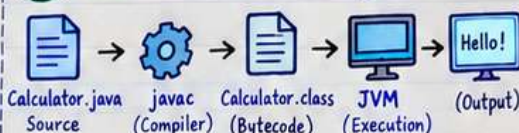
⑥ Easy Analogy

Think of Java like a restaurant:



- JDK** → Kitchen + tools to prepare the food
- JRE** → Everything needed to serve the food
- JVM** → Worker that actually performs the execution.

⑦ From Code to Output

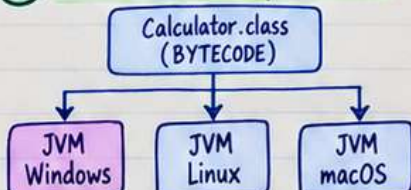


JDK helps create the program.
JVM executes the generated bytecode.

⑧ JDK vs JRE vs JVM

COMPONENT	PURPOSE	KEY IDEA
JDK	Development	Build Java applications
JRE	Runtime	Provides environment to run applications
JVM	Execution	Executes bytecode

⑨ Platform Independence



The bytecode can remain the **same** while the JVM implementation differs by platform.

SAME BYTECODE + **DIFFERENT JVM IMPLEMENTATIONS**

★ INTERVIEW SURPRISE

Q: Which one is responsible for executing Java bytecode?

A: **JVM**

Q: Can we develop Java applications with only a JVM?

A: **No.** The JVM executes bytecode, but development requires tools provided by the JDK.

JDK → Development tools **JVM** → Execution

★ Remember

- **JDK** → Build & develop
- **JRE** → Runtime environment
- **JVM** → Execute bytecode
- **JDK** contains the tools needed for development.
- **JVM** is the execution engine.

⚠ DON'T CONFUSE

- **JDK** is **NOT** the JVM.
- **JRE** is **NOT** the compiler.
- **JVM** does **NOT** compile .java source code.

.java → javac → .class → **JVM**

3. JAVA BASICS + SYNTAX

Learn the basic rules that every Java program follows.

1 Java Basics

- Java is a simple, **class-based**, **object-oriented** programming language.
- Java follows the **WORA** idea — Write Once, Run Anywhere.
- Java source code is written inside **.java** files.
- Java programs are compiled into **bytecode** before execution.
- Java is **case-sensitive**.
- A Java application is commonly organized using **classes** and **methods**.

2 Structure of a Java Program

```
class Demo {
    public static void main(String[] args) {
        int number = 10;
        System.out.println("Number = " + number);
    }
}
```

Class name
Main method (entry point)
A simple statement

Every Java program must have at least one class. Execution starts from the **main()** method.

3 Basic Syntax Rules

- Every statement ends with a **semicolon** (;).
- Class name should start with an uppercase letter.
- File name should be same as the public class name.
- Java is **case-sensitive** (sum and Sum are different).
- Code is written inside curly braces {}.
- Whitespace (spaces, tabs, new lines) is ignored.
- Comments are used to write notes in the code.

4 Comments in Java

- Single Line Comment**
// This is a single line comment
- Multi Line Comment**
/* This is a multi line comment
It can span multiple lines
*/
- Documentation Comment**
/** This is a documentation comment
*/

Comments are ignored by the compiler.

- ✓ Used to improve code readability.
- ✓ Helpful for documentation.
- ✓ Do not affect the program output.

5 Identifiers

- Identifiers are names given to classes, methods, variables, etc.
- Can contain letters (a-z, A-Z), digits (0-9) and underscore (_).
- Must start with a letter, _ or \$.
- Cannot be a reserved keyword.
- Should be meaningful.

Valid: studentName _count \$value

Invalid: 1name student-name class

6 Keywords

- Keywords are reserved words in Java.
- They have a special meaning and cannot be used as identifiers (class names, variable names, etc).

Some Java Keywords:

```
class public static void
int boolean if else
for while return new
this super final try
catch extends implements ...
```

7 Data Types (Primitive)

Java has 8 primitive data types.

Type	Size	Example
byte	1 byte	byte b = 5;
short	2 bytes	short s = 10;
int	4 bytes	int n = 100;
long	8 bytes	long l = 1000L;
float	4 bytes	float f = 3.14f;
double	8 bytes	double d = 3.14;
char	2 bytes	char ch = 'A';
boolean	1 bit	boolean flag = true;

8 Variables

- A variable is a named memory location used to store data.
- You must declare the type before using a variable.
- Syntax: **dataType** **variableName** = value;

Example:

```
int age = 20;
double price = 99.5;
char grade = 'B';
```

9 Constants

- A **constant** is a value that cannot be changed.
- In Java, we use the **final** keyword to create a constant.

Example:

```
final int MAX = 100;
final double PI = 3.14159;
```

Once a value is assigned to a final variable, it cannot be modified.

10 Naming Conventions

Type	Convention
Class Name	PascalCase (e.g., StudentInfo)
Method Name	camelCase (e.g., calculateSum)
Variable Name	camelCase (e.g., totalMarks)
Constant Name	UPPER_CASE (e.g., MAX_VALUE)
Package Name	lowercase (e.g., com.example)
Interface Name	PascalCase (e.g., Runnable)
File Name	Same as public class name

★ INTERVIEW SURPRISE

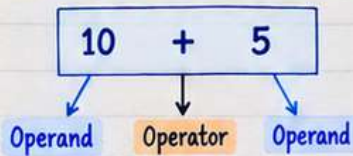
Q: Why is Java case-sensitive? **A:** Because the compiler treats uppercase and lowercase letters as different characters.

4. OPERATORS + EXPRESSIONS

Operators perform operations. Expressions combine values and operators to produce a result.

① What is an Operator?

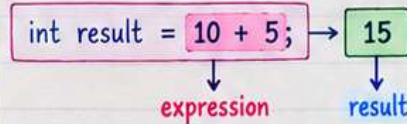
An **operator** is a symbol that tells Java to perform an operation on one or more operands.



1 Operands are the values on which an operator works.

② What is an Expression?

An **expression** is a combination of values, variables and operators that produces a result.



Java evaluates expressions according to operator **precedence** and **associativity**.

③ Types of Operators in Java

CATEGORY	OPERATORS	PURPOSE
Arithmetic	+ - * / %	Mathematical operations
Unary	++ -- + - !	Works with one operand
Relational	== != > < >= <=	Compares values
Logical	&& !	Combines conditions
Assignment	= += -= *= /= %=	Assigns/updates values
Bitwise	& ^ ~ << >> >>>	Works at bit level
Ternary	?:	Short conditional expression

1 Different operators have different **precedence**.

④ Arithmetic Operators

OPERATOR	OPERATION	EXAMPLE
+	Addition	10 + 5 = 15
-	Subtraction	10 - 5 = 5
*	Multiplication	10 * 5 = 50
/	Division	10 / 5 = 2
%	Remainder	10 % 3 = 1

IMPORTANT:

When both operands are integers:
10 / 3 → 3 (not 3.33)

⑤ Unary Operators

Unary operators work with a single operand.

OPERATOR	MEANING
+a	Unary plus
-a	Unary minus
++a	Pre-increment
a++	Post-increment
--a	Pre-decrement
a--	Post-decrement
!a	Logical NOT
~a	Bitwise complement

```
int a = 5;
int x = ++a;
a = 6
x = 6
```

PRE → change first

```
int b = 5;
int y = b++;
b = 6
y = 5
```

POST → use first

⑥ Relational Operators

OPERATOR	MEANING
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater or equal to
<=	Less or equal to

Example:

```
int a = 10;
int b = 20;
a < b → true
a == b → false
a > b → false
```

Result is boolean: true / false

⑦ Assignment Operators

OPERATOR	SAME AS
a = 10	-
a += 5	a = a + 5
a -= 5	a = a - 5
a *= 5	a = a * 5
a /= 5	a = a / 5
a %= 5	a = a % 5

⑧ Logical Operators

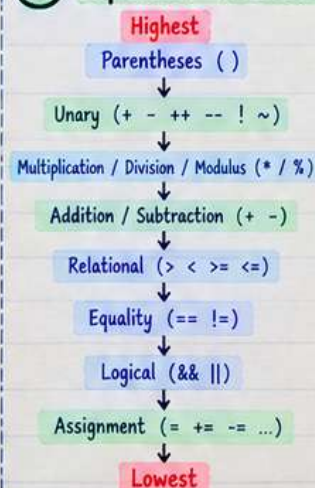
OPERATOR	MEANING	DESCRIPTION
&&	AND	Both conditions must be true
	OR	At least one condition must be true
!	NOT	Reverses the boolean result

Example:

```
int age = 25;
boolean isValid = age >= 18 && age <= 60;
```

```
true && true → true
true && false → false
true || false → true
!true → false
```

⑨ Operator Precedence



Example:

```
int result = 10 + 2 * 3;
First: 2 * 3 = 6
Then: 10 + 6 = 16
result = 16
```

Use parentheses when you want to make the order obvious.

Code Example

```
int a = 10;
int b = 3;
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a / b); // 3
System.out.println(a % b); // 1
System.out.println(a > b); // true
```

★ INTERVIEW TIP

Q: What is the difference between an operator and an expression?

A: **Operator** → symbol that performs an operation.
Expression → combination of values/operators that produces a result.

Understanding precedence helps you predict expression results.

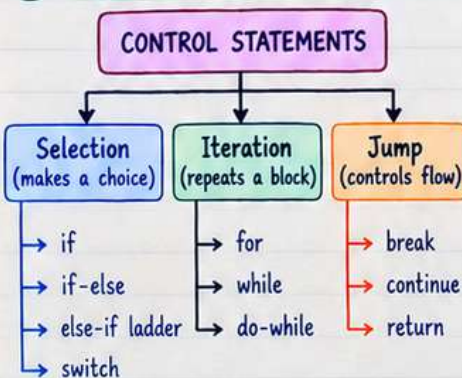
★ REMEMBER

- Arithmetic → Calculate
- Relational → Compare
- Logical → Combine conditions
- Assignment → Assign/update
- Unary → One operand
- Precedence → Decides evaluation order

5. CONTROL STATEMENTS

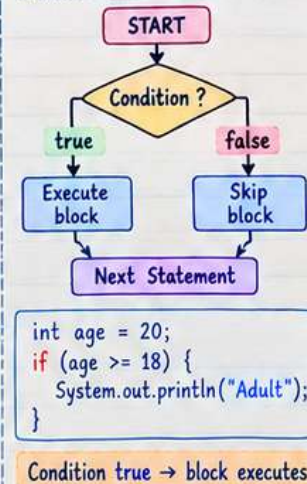
Control statements decide which code executes and how many times it executes.

① Types of Control Statements



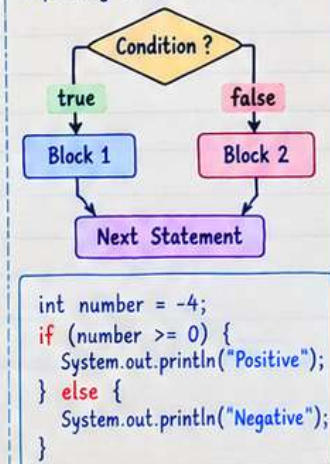
② if Statement

Executes a block only when the condition evaluates to true.



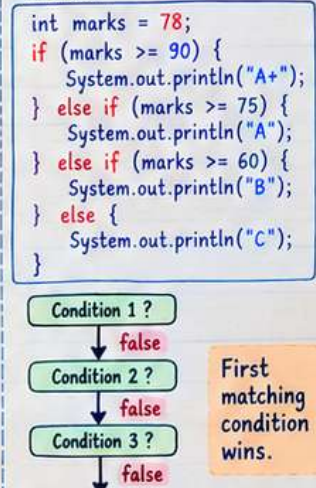
③ if-else Statement

Chooses one of two blocks depending on the condition.



④ else-if Ladder

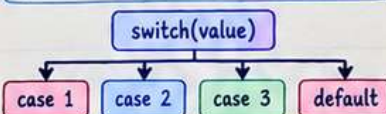
Useful when multiple conditions must be checked in order.



⑤ switch Statement

Useful when one expression is matched against multiple fixed choices.

```
int option = 2;
switch (option) {
    case 1:
        System.out.println("Start");
        break;
    case 2:
        System.out.println("Settings");
        break;
    case 3:
        System.out.println("Exit");
        break;
    default:
        System.out.println("Invalid");
}
```



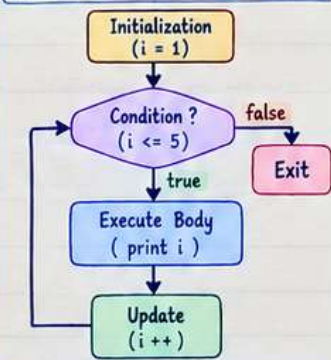
! break prevents fall-through to the next case.

⑥ for Loop

Best suited when the number of iterations is known.

for (initialization; condition; update)

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

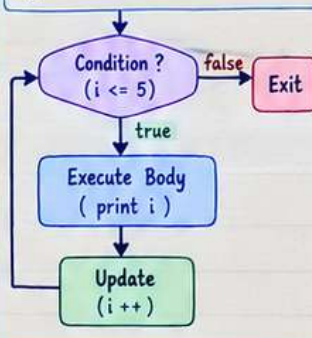


Initialization → Condition → Body → Update (repeats)

⑦ while Loop

Repeats a block while the condition remains true.

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

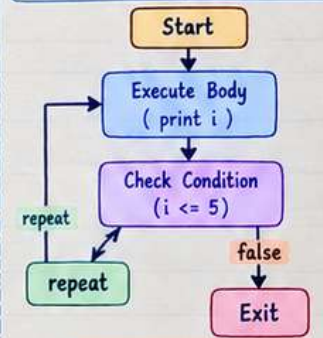


Condition checked BEFORE the body.

⑧ do-while Loop

Executes the body first and checks the condition afterward.

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while (i <= 5);
```



Runs AT LEAST ONCE.

⑨ Jump Statements

STATEMENT	PURPOSE	EXAMPLE
break	Exits loop or switch.	if (i == 3) break;
continue	Skips current iteration.	if (i == 3) continue;
return	Exits method (optional value).	return x;

Examples

break example:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.print(i + " ");
}
```

Output: 1 2

continue example:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
```

Output: 1 2 4 5

🔗 INTERVIEW TIP

Q: Which loop always executes at least once?

A: do-while

Why? Because the condition is checked AFTER the loop body.

Q: Which loop is commonly preferred when the iteration count is known?

A: for loop.

★ **REMEMBER**

- if → one condition
- if-else → two paths
- else-if → multiple conditions
- switch → multiple fixed choices
- for → known iterations
- while → condition first
- do-while → body first
- break → exit
- continue → skip current iteration
- return → exit method

6. METHODS + PARAMETERS

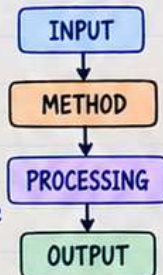
Break a large task into small, reusable pieces.

① What is a Method?

A method is a named block of code designed to perform a specific task.

Methods improve:

- ✓ Reusability
- ✓ Readability
- ✓ Modularity
- ✓ Testing and maintenance



② Method Syntax

```

returnType methodName(parameters) {
    // statements
    return value;
}
  
```

Annotations:
 returnType: type of value returned
 methodName: name used to call the method
 parameters: inputs received by the method
 return value: sends a value back

A void method does not return a value.

③ Example

```

static int multiply(int a, int b) {
    return a * b;
}
  
```

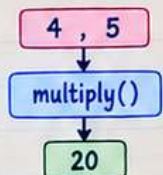
Method Definition

```

int result = multiply(4, 5);
  
```

Method Call

Output: 20



④ Parameters vs Arguments

```

static int add(int a, int b)
          ^   ^
          a   b
    Parameters (in method definition)

add(10, 20);
   ^   ^
   10  20
    Arguments (in method call)
  
```

TERM	MEANING
Parameter	Variable in method definition.
Argument	Actual value passed during method call.

⑤ Types of Methods

- Static Method**
 - Belongs to the class.
 - Example:** Math.max(10, 20);
- Instance Method**
 - Called using an object.
 - Example:** student.display();
- Parameterized Method**
 - Receives input.
 - Example:** add(10, 20);
- Void Method**
 - Returns no value.
 - Example:**

```
void display() {
    System.out.println("Hello");
}
```

⑥ Method Overloading

Multiple methods can have the same name when their parameter lists are different.

```

int add(int a, int b)
double add(double a, double b)
int add(int a, int b, int c)
  
```

Same Method Name + Different Parameters
↓
Method Overloading

⚡ IMPORTANT:

Changing only the return type does NOT create a valid overload.

```

int add(int a, int b)
double add(int a, int b)
  
```

✗ Not allowed

⑦ How Java Passes Values

Java is always **pass-by-value**:

```

int a = 10;
change(a);
System.out.println(a);

void change(int value) {
    value = 20;
}
  
```

Original a = 10 (unchanged)

The method receives a copy of the value.

When an object is passed, the copied value is the reference to that object.

JAVA = PASS-BY-VALUE

⑧ Recursion

Recursion occurs when a method calls itself to solve a smaller version of the same problem.

```

factorial(4)
  ↓
4 × factorial(3)
  ↓
4 × 3 × factorial(2)
  ↓
4 × 3 × 2 × factorial(1)
  ↓
24
  
```

```

static int factorial(int n) {
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}
  
```

Base case (n == 0)

Recursive case (factorial(n-1))

⑨ Recursion Rule

Base Case

Stops the recursive calls.

Recursive Case

Moves the problem toward the base case.

⚠ Without a proper base case, recursion can continue until **StackOverflowError**.

⑩ Why Use Methods?

- ✓ Avoid repeating code
- ✓ Divide large problems into smaller tasks
- ✓ Improve readability
- ✓ Make testing easier
- ✓ Make programs easier to maintain

★ INTERVIEW SURPRISE

Q: Is Java pass-by-reference?

A: No. Java is always pass-by-value. For objects, the value being passed is a copy of the reference.

Q: Can changing only the return type overload a method?

A: No. The parameter list must differ. Method overloading is based on the number, type or order of parameters.

★ REMEMBER

- Method → Reusable block of code
- Parameter → Variable in method definition
- Argument → Actual value passed
- Overloading → Same name + different parameter list

- Recursion → Method calls itself
- Base Case → Stops recursion
- Java → Always pass-by-value

7. ARRAYS IN JAVA

Store multiple values of the same type using one variable.

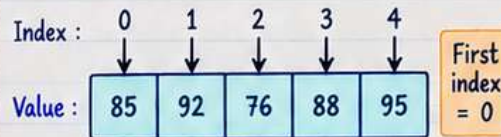
① What is an Array?

An array is a **FIXED-SIZE** collection that stores multiple values of the **SAME TYPE**.

- Elements are stored in indexed positions.
- Indexing starts from 0.
- Array length is fixed after creation.
- Arrays can store primitive values or object references.

② Array Structure

```
int[] marks = {85, 92, 76, 88, 95};
```



Every element is accessed using its index.

```
marks[0] → 85
marks[2] → 76
marks[4] → 95
```

③ Declaring an Array

1 Declaration: `int[] numbers;`

2 Creation: `numbers = new int[5];`

3 Declaration + creation:

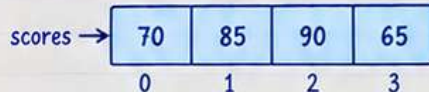
```
int[] numbers = new int[5];
```

- new int[5] creates space for 5 integers.

length = 5
valid indexes: 0 1 2 3 4
Last index = length - 1

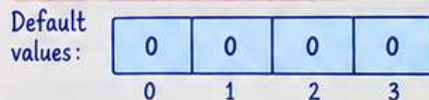
④ Initializing an Array

```
int[] scores = {70, 85, 90, 65};
```



- Java automatically determines the array size from the values.

```
int[] values = new int[4];
```



⑤ Accessing Elements

```
int[] nums = {10, 20, 30, 40};
System.out.println(nums[1]);
```

Output: 20

```
nums[3] = 100;
```

Now: [10, 20, 30, 100]

- Array elements can be read and updated using their index.

⚠️ `nums[4]` is invalid because the last valid index is 3.

⑥ Finding Array Length

```
int[] nums = {10, 20, 30, 40};
System.out.println(nums.length);
```

Output: 4

- For arrays, length is a field — use `nums.length`.
- It gives the total number of elements.
- Do NOT use `nums.length()`.

ARRAY
→ length

STRING
→ length()

⑦ Traversing an Array

① Using normal for loop (with index)

```
int[] nums = {10, 20, 30, 40};
for (int i = 0; i < nums.length; i++) {
    System.out.println(nums[i]);
}
```

② Using enhanced for loop (for-each)

```
for (int value : nums) {
    System.out.println(value);
}
```

for loop → useful when index is needed

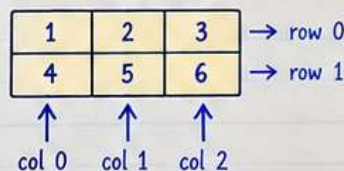
for-each → convenient when only values are needed

⑧ Types of Arrays

TYPE	IDEA	EXAMPLE
1D Array	Linear collection	<code>int[] arr</code>
2D Array	Rows + columns	<code>int[][] matrix</code>
Jagged Array	Rows can have different lengths	<code>int[][] data</code>

2D Array Example:

```
int[][] matrix = {{1, 2, 3},
                  {4, 5, 6}};
```



⑨ Important Points

- ✓ Array size is fixed.
- ✓ Index starts from 0.
- ✓ Last index = length - 1.
- ✓ Invalid index can cause **ArrayIndexOutOfBoundsException**.
- ✓ Arrays are objects in Java.
- ✓ Use `.length` to get array size.

★ INTERVIEW SURPRISE

Q: What happens if we access an invalid array index?

```
int[] a = {10, 20, 30};
System.out.println(a[3]);
```

A: **ArrayIndexOutOfBoundsException**
Because valid indexes are: 0, 1, 2.

★ REMEMBER

- Array → Fixed-size collection
- Index → Starts at 0
- Last index → length - 1

- Size → `array.length`
- Traversal → for / for-each

8. STRINGS + STRING METHODS

Strings are objects used to represent sequences of characters.

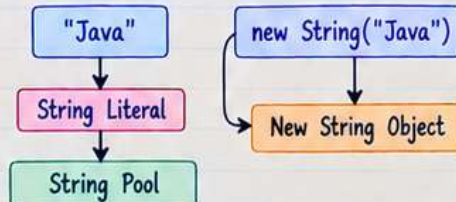
① What is a String?

A String represents a sequence of characters.

- String is a class in Java.
- String objects are **immutable**.
- String literals can be stored in the **String Constant Pool**.
- Strings are widely used in Java applications.

② Creating a String

```
String s1 = "Java"; // String literal
String s2 = new String("Java");
```



Both represent the text Java, but they are created differently.

③ Strings are Immutable

```
String name = "Java";
name.concat(" Programming");
System.out.println(name);
```

Output:
Java

String methods do not modify the original String object. They return a new String when a changed value is needed.



ORIGINAL STRING → UNCHANGED

④ Comparing Strings

```
String a = new String("Java");
String b = new String("Java");
```

a == b → **false**

a.equals(b) → **true**

== → compares object references

equals() → compares String content

REFERENCE → ==

CONTENT → equals()

⑤ Important String Methods

METHOD	PURPOSE	EXAMPLE
length()	number of characters	"Java".length() → 4
charAt(index)	character at index	"Java".charAt(1) → 'a'
substring(b, e)	extracts part of a String	"Java".substring(1, 3) → "av"
toUpperCase()	converts to uppercase	"java".toUpperCase() → "JAVA"
toLowerCase()	converts to lowercase	"JAVA".toLowerCase() → "java"
trim()	removes leading/trailing spaces	" Java ".trim() → "Java"
contains()	checks whether text exists	"Java".contains("av") → true
startsWith()	checks starting text	"Java".startsWith("Ja") → true
endsWith()	checks ending text	"Java".endsWith("va") → true
indexOf()	finds first matching index	"Java".indexOf('a') → 1

⑥ Joining Strings

```
String first = "Code";
String second = "Govind";
String result = first + " " + second;
```

Output: Code Govind

```
System.out.println("Java" + 10);
```

Output: Java10

When a String is involved with +, values are combined as text.

⑦ Escape Sequences

SEQUENCE	MEANING
\n	New line
\t	Tab space
\"	Double quote
\\	Backslash

Example:

```
System.out.println("Java\nProgramming");
```

Output:

Java
Programming

⑧ String Practice Example

```
String text = "Java Programming";
System.out.println(text.length());
System.out.println(text.charAt(0));
System.out.println(text.toUpperCase());
System.out.println(text.substring(5));
System.out.println(text.contains("Program"));
```

Output:

16
J
JAVA PROGRAMMING
Programming
true

⑨ Important Points

- ✓ String is immutable.
- ✓ String is a class, not a primitive type.
- ✓ String literals can use the String Constant Pool.
- ✓ == checks references.
- ✓ equals() checks content.
- ✓ String methods generally return a String (or other result) rather than modifying the original String.

★ INTERVIEW SURPRISE

Q: Why are Strings immutable in Java?

A: Immutability makes Strings safer to share and supports efficient reuse, especially with String literals and the String Constant Pool.

Q: What should you normally use to compare String content?

A: equals() == → reference
 equals() → content

★ REMEMBER

- String → Sequence of characters
- Immutable → Original String does not change
- length() → Number of characters

- charAt() → Character at index
- substring() → Extract part
- equals() → Compare content